Communic-aid

Adapting Dwell Time and Confirmation in Gaze-Based Messaging with a Commodity iPhone

CS 435 Human-Computer Interaction, Group Course Project, Assignment 3: High-Fidelity Prototyping and User Evaluation
Group TO FILL: group number: Vandya Bisaria, Mansie Singh, Sinngam Khaidem
Instructor: Dr. Yogesh K. Meena, IIT Gandhinagar
November 2026

Summary. Communic-aid is a working, software-based high-fidelity prototype of a hands-free messaging app for people with motor impairments. An iPhone with a TrueDepth (Face ID) camera tracks gaze, head pose, blinks and eyebrow raises, and listens for the word "go". A Python core on a computer turns these into selections in a browser-based chat app with messages, calls and a gaze keyboard. The core idea is that the interface learns how much evidence to ask for before it acts, per person, per context and per level of risk: a dwell learner sets how long a look must last, and a modality router decides which confirmation signal to ask for on risky actions. We evaluated the prototype in a controlled usability test with two participants. Code, README with run instructions, the demo video and the evaluation data are in the project repository: TO FILL: public GitHub link.

1 System Design and Architecture

1.1 Problem and goal

For many people with ALS, muscular dystrophy, cerebral palsy or spinal cord injury, the eyes remain controllable long after the hands. Gaze interfaces suffer from the Midas touch problem [3]: the eyes are always looking at something, so the interface cannot tell "do this" from "I am reading". The usual remedy is a *dwell* time: a button is selected only after the eyes have stayed on it long enough. A long dwell is safe but slow; a short dwell is fast but selects things by mistake. Adaptive dwell methods tune one threshold for all actions [5, 7], and multimodal methods add a fixed second signal [2, 8]. Our earlier reports identified three gaps: confirmation does not scale with the cost of an error, dwell adaptation rarely learns from implicit signals, and no system chooses the confirmation modality from experience. Communic-aid addresses all three with one question: *how much evidence should the interface ask for before it acts?*

1.2 Architecture

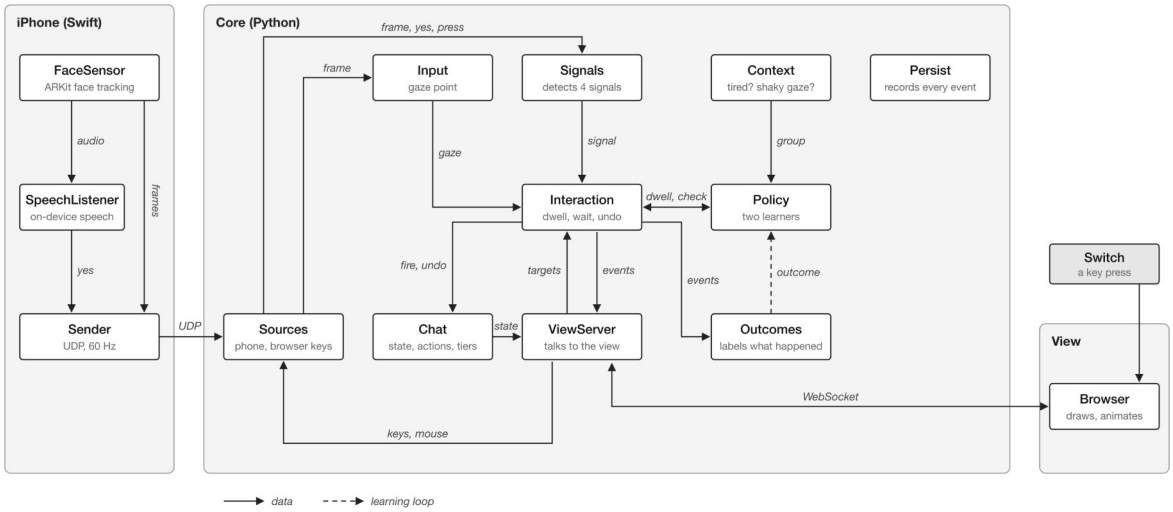


Figure 1. Architecture. The iPhone only senses, the Python core makes every decision, and the browser only draws. The dashed arrow is the learning loop: outcomes update the policy.

The system has three parts (Figure 1):

1. **iPhone app** (Swift, ARKit face tracking). Sends, 60 times a second over UDP, the gaze ray and eye position in camera space, the nose direction, and blink and eyebrow blendshapes. On-device speech recognition listens only for the word “go”. A live face preview helps the person position themselves.
2. **Core** (Python, asyncio). Maps gaze to the screen with a nine point calibration (quadratic least squares with a small ridge term), smooths it with an exponential filter, holds it still during blinks, and corrects slow drift from the centres of buttons just selected. It detects signals with per-person thresholds set halfway between rest and peak in a signal check (blink held 500 ms, eyebrows raised 300 ms, the spoken word, or a switch key). It runs the dwell and confirmation logic, labels outcomes, updates the learners and logs every event.
3. **View** (HTML, CSS and JavaScript over a WebSocket). Draws the screens and animations and holds no logic.

There is no external API: all logic runs locally, so face and gaze data never leave the two devices. Table 1 lists the technologies.

Table 1. Technologies used.

Part	Technology
Sensor app	Swift, SwiftUI, ARKit face tracking, Apple Speech framework, UDP
Core	Python 3.11, asyncio, aiohttp (web and WebSocket server), NumPy
View	HTML, CSS, JavaScript in a full screen browser window
Analysis	Python scripts with Matplotlib: heatmaps, dwell plots, summaries
Testing	pytest, 79 automated tests; a mouse mode that runs without the phone

1.3 Risk tiers

Every action belongs to a risk tier whose accident cost doubles with each step (Table 2). Both learners use this cost.

Table 2. Risk tiers. Tier 2 and 3 add a confirmation step after a fixed 1000 ms dwell.

Tier	Actions	Accident cost	Confirmation
0	Open a chat, back, scroll, navigation	1	Learned dwell
1	Letters, word suggestions, Space, Backspace, settings	2	Learned dwell
2	Send, react, end a call	4	Router: dwell alone or a signal
3	Call, delete, forward	8	Always a signal chosen by the router

1.4 Implicit outcomes

The learners never ask the person for feedback. The core labels each look and each confirmation request from what the person does next (Table 3). Statistics are kept per context group: rested or tired (over 20 minutes in, or evening) crossed with steady or shaky gaze (recent gaze spread over 1.5 times a steady baseline).

Table 3. Implicit outcomes.

Outcome	Rule	What it tells the learner
Accident	A selection undone within 5 s, or a letter deleted within 3 s	The dwell was too short
Worked	A selection that was not undone	The dwell was long enough
Give-up	A look filled at least half the dwell, then the same button was chosen afresh within 5 s	The dwell was too long
Stray glance	Any other look that did not fill	Normal reading; how long glances last

Outcome	Rule	What it tells the learner
Blocked	A confirmation request timed out with no attempt	The check stopped an unwanted action

1.5 Learning

Dwell learner (tiers 0 and 1). From counts of intended and unintended looks, stray glances and give-ups, it draws by Thompson sampling [9] the probability π that a look is intended, a mean stray glance length g and a mean patience p, and scores each dwell d from 800 to 2000 ms:

$$V(d) = \pi\left[(1 - G(d))(1 - \lambda d) + r_{\text{give-up}} G(d)\right] - (1 - \pi) A(d) c_{\text{tier}}$$

with $A(d) = e^{-d/g}$ the chance a stray glance fires, $G(d) = 1 - e^{-d/p}$ the chance the person gives up first, $\lambda = 0.6$ per second and $r_{\text{give-up}} = -0.5$. In words: a longer dwell stops more stray glances from firing, but makes the person wait and sometimes give up; the learner picks the dwell with the best balance, and a higher tier cost pushes it towards caution. After each selection the dwell moves 5% towards the best d, by at most 10 ms when shortening and up to 100 ms when lengthening. It starts at 1500 ms and holds for 20 selections per group.

Modality router (tiers 2 and 3). A contextual bandit [4] keeps, per signal, tier and group, counts of worked, give-up, blocked and accident outcomes, samples their probabilities from a Dirichlet distribution and asks for the signal with the highest expected reward. A worked signal earns 1 minus its effort, which grows with the person's time to produce it; a blocked accident earns a small reward; an accident costs the tier cost. The person's preferred signal starts with a bonus. In short, the router learns which confirmation works best for each person and context, at the lowest effort.

2 Compromises in Prototyping

We built the prototype wide enough to show every screen of a messaging app (horizontal), and deep along the one path that carries our research question: from the camera to the learners (vertical). Table 4 compares the two.

Table 4. Horizontal and vertical scope of the prototype.

	Horizontal prototype (breadth)	Vertical prototype (depth)
Scope	Every screen: onboarding, signal check, calibration, chat list, thread, compose, call, message actions, settings, typing test	The full path from sensing to learning, working end to end with real users
What is real	Navigation, layout, feedback, undo, tier colours, both keyboards, word suggestions and quick replies	Gaze and head pointing, signal detection, dwell and confirmation, outcome labelling, both learners, per-person saving, logging and analysis
What is simulated	Contacts, chats and replies come from a local file; replies arrive after 7 s; calls connect after 3 s with no audio	Nothing on this path is simulated
What it tests	Whether the layout, navigation and feedback hold together	Whether risk tiers and adaptive dwell work for a real task

Other compromises we made:

- The phone is a sensor for a computer screen rather than a standalone app. This keeps targets large and lets us log everything, but adds a second device.
- An iPhone with Face ID is required, and apps signed with a free developer account expire every 7 days.
- Speech recognises one word, "go", rather than dictation, which our interviewee with slurred speech found unreliable.
- Word suggestions come from a fixed word list, not a language model, so no AI API is needed.
- Head pointing is built but not yet evaluated, and the tired flag uses a simple rule rather than a fatigue measure.

3 Conceptual Design

3.1 Metaphor

Communic-aid borrows the **messaging app** people already know: chats, threads, a call button, a draft box and Send, so no new model of communication must be learned. Selection follows a **press and hold** metaphor: looking at a button presses it, and the button fills like a progress bar until it fires, so a look can always be abandoned before it counts. Risky buttons add a **safety catch**, like the second step of a slide to unlock: after the fill, the button shows the signal it needs (an eye, an eyebrow, "go"), and nothing happens without it.

3.2 Conceptual model

The objects are chats, messages, contacts and a draft; the actions are open, type, send, react, call, delete, forward and undo. Each action carries a tier, and the tier, not the screen, decides how much evidence it needs. The user controls what to do, which signals are on and which comes first; the system controls how long to wait and when to ask for a signal, and adjusts both from how the person uses it. Every action can be undone for 5 seconds, and losing the face never fires anything.

3.3 Design principles

From our interviews with three people with motor impairments and the literature:

1. Calm and low effort: nothing rushes or startles the person.
2. Every visual element gives feedback or prevents a mistake.
3. The cost of an error decides the check.
4. Learn from implicit behaviour, never from ratings.
5. Reading is safe: text is never a target, only buttons are.

3.4 Design space

Table 5 lists the dimensions we explored and our choices. The choices favour mainstream hardware [12, 6] and per-person adaptation, in the spirit of ability-based design [11].

Table 5. Design space.

Dimension	Options considered	Chosen
Sensor	Eye tracker, webcam, iPhone TrueDepth	iPhone TrueDepth: cheap, mainstream, depth based
Pointing	Gaze, head, gaze and head	Gaze, with head pointing as an option
Selection	Dwell, blink, smooth pursuit, gaze gestures	Dwell with a learned time
Confirmation	None, fixed second signal, user chosen, learned	Learned per tier and context
Signals	Blink, eyebrows, voice, switch, EMG, BCI	Blink and hold, eyebrows, "go", switch
Adaptation	Explicit ratings, implicit outcomes	Implicit outcomes
Text entry	QWERTY, grouped letters, dwell-free, AI text	Grouped (default) and QWERTY, with suggestions and quick replies
Feedback	Visual, sound, colour coding	All three

4 Prototype Description and Functionalities

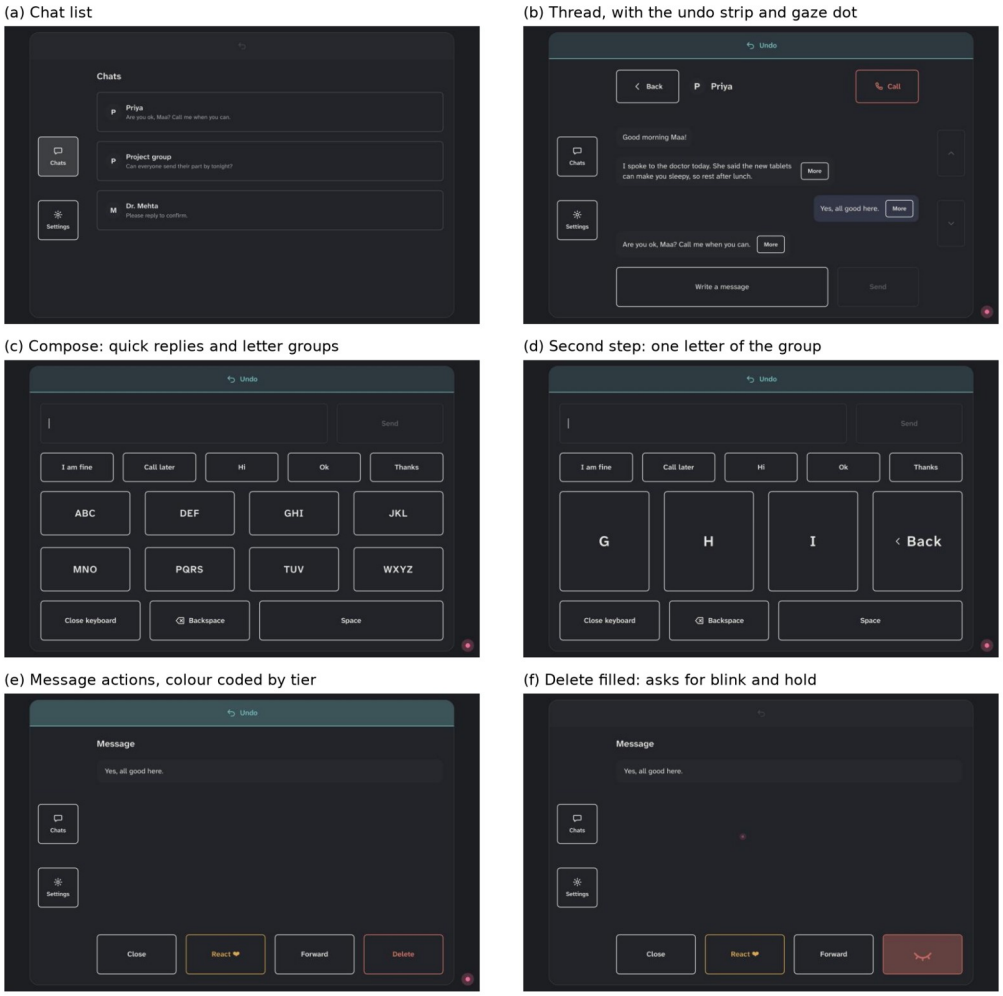


Figure 2. Screens of the working prototype. In (f) the Delete button has filled and now shows the closed eye: it waits for blink and hold before it deletes.

Figure 2 shows the main screens. The core interaction flow is:

1. Look at a button. It fills as long as the look stays on it.
2. When it fills, a tier 0 or 1 action happens at once.
3. On a tier 2 or 3 button, the button shows the signal it needs, and the person gives it (or looks away to cancel).
4. After any action, an undo strip appears for 5 seconds.

Navigation. Chats and Settings sit in a fixed side bar; Back, scroll buttons and the undo strip are in the same places on every screen. A chat opens a thread; More on a message opens its actions (React, Forward, Delete); Call opens a call screen with End call.

Text entry. The grouped keyboard has eight letter groups (abc to wxyz): a group, then a letter (Figure 2c, d). QWERTY is one selection per letter. Both offer word suggestions and five quick replies.

Feedback and transitions. A pink dot shows where the system thinks the person is looking. Buttons fill as they are looked at and glow when they fire; tier 2 buttons have an amber outline and tier 3 a red one. A confirmation request shows the signal's icon on the button and waits up to 4 s; a wrong signal makes the button shake and plays a soft error tone, and a confirmed one plays a tick. Plain language banners appear when tracking fails ("I can't see your eyes. Look at the screen.", "Move a little closer"), only after the problem has lasted a second, so they never flicker.

Setup and settings. First use asks three onboarding questions, then runs the signal check and calibration (redone automatically if the error exceeds 100 px). Settings turn signals on or off, choose the one to try first, switch sounds and keyboard, and rerun calibration or the signal check. Three switch presses always escape to safety.

Study tools. A typing test mode shows a passage and measures words per minute for two minutes from the first letter. A terminal log shows every learner decision live. Scripts produce per-person heatmaps, dwell plots and summary tables.

5 User Evaluation and Results

5.1 Evaluation method and data gathering

We ran a **task-based usability test** in a **controlled setting**: a quiet room, the laptop on a desk, the phone mounted below the screen at 40 to 60 cm, the participant's head free to move, and an experimenter present. Data came from four sources:

- the system's event log: every look, selection, outcome and learner update;
- a two minute typing test, run three times with each keyboard;
- the System Usability Scale (SUS) [1], filled in after the session;
- open feedback from each participant.

5.2 Participants

Two able-bodied adults (P1, P2) have completed the test, using gaze pointing with the adaptive learners. Two more able-bodied participants and one participant with a motor impairment are scheduled.

5.3 Tasks

After the signal check and calibration, each participant used the app freely and then completed the tasks in Table 6. Free use lasted 9 minutes for P1 and 24 for P2.

Table 6. User tasks.

#	Task	Done when
1	Open the chat with Priya, go to Settings and change the preferred signal, go back to the chats	Chat list showing, new signal saved
2	In Priya's chat, type "see you at five" and send it	The message is sent
3	Call Priya; when the call connects, end it	Back in Priya's chat
4	Delete the message "Yes, all good here."	The message is gone
5	Typing test: copy a passage for two minutes, three times with each keyboard	Words per minute shown

5.4 Results

Table 7. Free use session, from the event log. Selections exclude undo and redo.

Measure	P1	P2
Calibration error	80 px	69 px
Selections (tier 0 / 1 / 2 / 3)	51 (18 / 29 / 2 / 2)	95 (32 / 51 / 7 / 5)
Accidents	2 (both tier 1, both undone)	0
Dwell give-ups	6	11
Stray glances	145	541
Confirmation requests on tiers 2 and 3 (worked / gave up)	5 (4 / 1)	13 (12 / 1)
Dwell at end, tier 0 / tier 1	1216 / 1333 ms	1219 / 1431 ms

Accidents and confirmation. Across 146 selections there were two accidents, both letters, both undone (Table 7). No tier 2 or 3 action fired by mistake, and 16 of 18 confirmation requests succeeded. The router adapted within each session: when P1 gave up on raised eyebrows to confirm Send, the next Send asked for blink and hold, which worked, as did the call and the deletion that followed. P2 began with blink and hold; after three successful confirmations the router accepted dwell alone on tier 2 (four times, all worked), and after P2 made "go" the preferred signal, P2 confirmed it five times out of five.

Dwell learning. Stray glances outnumbered selections three to six times: a gaze interface produces much implicit evidence even when nothing is selected. After its warm-up the learner shortened both tiers steadily (Figure 3). Tier 0 fell from 1500 to about 1220 ms for both participants. Tier 1 stayed longer, as expected from its higher cost.

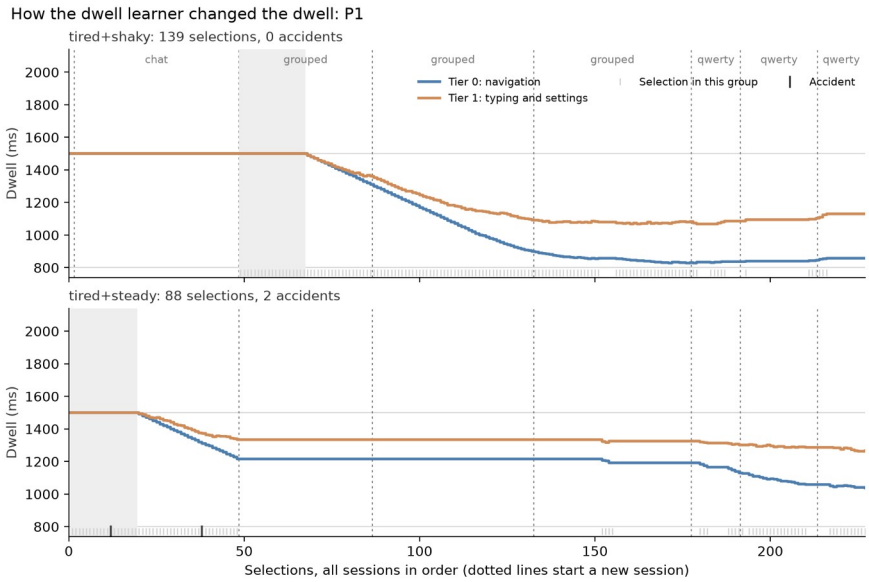


Figure 3. P1's dwell over all sessions, one panel per context group, one line per tier. The first session is free use, then three grouped and three QWERTY typing runs. Shading marks the warm-up; marks at the bottom are selections (light) and accidents (dark).

Typing. The grouped keyboard was 1.5 (P1) and 2.8 (P2) times faster than QWERTY (Table 8), despite almost two selections per letter. Target size explains it: a letter group was 316 × 113 px and a QWERTY key 113 × 67 px, while the calibration error was 69 to 80 px, about one QWERTY key. We report keystrokes per character as in [10].

Table 8. Typing test, mean of three two minute runs per keyboard.

Participant	Keyboard	Words per minute	Accuracy	Keystrokes per character
P1	Grouped	2.23	100%	1.84
P1	QWERTY	1.50	91.7%	1.06
P2	Grouped	1.60	100%	1.86
P2	QWERTY	0.57	100%	1.11

Usability. Table 9 gives the SUS scores (0 to 100; about 68 is average). Both scores are below average: P1 scored 52.5 and P2 40.0. Both rated the functions as well integrated (4 of 5) and the system as not inconsistent (2 of 5), so the structure of the app held up. The low items were about effort: neither would yet like to use it frequently or felt very confident, and P2 felt they would need technical support and had a lot to learn before getting going (4 of 5 on both). The feedback below points to pointer jitter and the QWERTY keyboard as the main causes.

Table 9. System Usability Scale answers, from 1 (strongly disagree) to 5 (strongly agree). Odd items are positive statements (Q1 use frequently, Q3 easy, Q5 well integrated, Q7 quick to learn, Q9 confident) and even items negative (Q2 complex, Q4 need support, Q6 inconsistent, Q8 cumbersome, Q10 much to learn).

	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	SUS
P1	1	3	3	2	4	2	2	2	2	2	**52.5**
P2	2	3	2	4	4	2	2	3	2	4	**40.0**

5.5 User feedback

We wrote down each participant's comments after the session. They agreed on three points:

- **The pointer is too shaky.** P1 asked us to "add a smoothing factor just like mouse pointer speed", that is, a setting the person can tune. P2 also named shaky gaze as the first problem. The pointer currently uses an exponential filter with one fixed setting in the config file.
- **Corners are hard to reach.** Both said targets near the screen corners were difficult to point at. Gaze estimates from a camera below the screen are usually least accurate near the edges, and the calibration error of 69 to 80 px leaves little room for a target in a corner.
- **QWERTY is tiring.** P1 struggled with the QWERTY keyboard and P2 said it was "tough to focus" on. This matches Table 8: its keys are about the size of the calibration error.

P1 also said that coordinating head and gaze "needs getting used to", and P2 found deleting a message (task 4) harder than the other tasks. P2 wore glasses during the session.

These comments give three changes for the next version: a smoothing slider in Settings, moving targets away from the corners (or giving edge targets larger hit areas), and making the grouped keyboard the default.

5.6 Limitations

So far there are two able-bodied participants and no fixed baseline, so the results show that the learners behave as intended, not that adaptation beats a fixed dwell. The typing runs were recorded while the dwell was still learning, with grouped always first; the typing test now uses a fixed dwell, and the order will be counterbalanced. The remaining three participants will use the app twice, adaptive and fixed (1000 ms dwell, dwell alone on tier 2, the preferred signal on tier 3), in counterbalanced order, with SUS after each.

6 Ethics and Risk Assessment

6.1 Risks and mitigation

Table 10. Risks to participants and how we reduce them.

Risk	Mitigation
Fatigue and eye strain	Breaks offered between parts; the person can pause or stop at any time; the dwell lengthens after errors instead of rushing the person
Discomfort (neck, posture)	Head free to move; phone placed at a comfortable distance; banners ask the person to move only when tracking fails
Frustration from errors	Undo on every action for 5 s; risky actions need a second signal; a hint after 30 s of being stuck; no time pressure except in the typing test
Privacy of face and voice	Camera images are shown live only and never stored; speech is recognised on the phone and only the word "go" is sent; nothing goes to a cloud service
Identification from data	Data is saved under a participant code (P1, P2); names are kept apart from the data; shared evaluation data uses codes only

An experimenter supervised every session. The logs hold gaze coordinates, head position and angles, and interaction events; they hold no images, audio or names. The onboarding screen offers an age range and a condition category; participants could pick any option, these answers served as selection practice, and they were not analysed.

6.2 Statement of voluntary participation

Participation is voluntary. Each participant is told what the session involves and what is recorded, gives consent before starting, and may stop at any time without giving a reason. No sensitive personal data (names, contact details, images, audio or health records) is collected. The data is stored on the team's computer under participant codes and used only for this course project.

7 Conclusion

Communic-aid is a working high-fidelity prototype in which one consumer phone provides both pointing and confirmation for hands-free messaging, and the interface learns from ordinary use, without asking, how long to wait and which signal to ask for before acting. Early tests show rare, cheap, undoable accidents, no accidents on risky actions, a dwell that shortens within one session, and a router that adapts to each person. Usability scores were below average (SUS 52.5 and 40.0), mainly because of pointer jitter, hard to reach corners and the small QWERTY keys; these point to a tunable smoothing setting, edge friendly layouts and the grouped keyboard as the default. The remaining sessions will compare adaptive against fixed settings and include a participant with a motor impairment.

References

[1] John Brooke. 1996. SUS: A "Quick and Dirty" Usability Scale. In *Usability Evaluation in Industry*. Taylor & Francis, 189-194.

[2] Y. Cha, D. Chun, and S. Park. 2026. iPhoneme: Brain-to-Text Communication for ALS Using ConformerXL Decoding. arXiv:2604.16441.

[3] Robert J. K. Jacob. 1990. What You Look at is What You Get: Eye Movement-Based Interaction Techniques. In *Proceedings of CHI '90*. ACM, 11-18.

[4] Lihong Li, Wei Chu, John Langford, and Robert E. Schapire. 2010. A Contextual-Bandit Approach to Personalized News Article Recommendation. In *Proceedings of WWW '10*. ACM, 661-670.

[5] Päivi Majaranta, Ulla-Kaija Ahola, and Oleg Špakov. 2009. Fast Gaze Typing with an Adjustable Dwell Time. In *Proceedings of CHI '09*. ACM, 357-360.

[6] Yogesh Kumar Meena and M. Salvi. 2025. Multimodal Appearance-Based Gaze-Controlled Virtual Keyboard with Synchronous-Asynchronous Interaction for Low-Resource Settings. arXiv:2508.16606.

[7] Martez E. Mott, Shane Williams, Jacob O. Wobbrock, and Meredith Ringel Morris. 2017. Improving Dwell-Based Gaze Typing with Dynamic, Cascading Dwell Times. In *Proceedings of CHI '17*. ACM.

[8] Omar Namnakani, Yasmeen Abdrabou, Jonathan Grizou, and Mohamed Khamis. 2025. GazeBlend: Exploring Paired Gaze-Based Input Techniques for Navigation and Selection Tasks on Mobile Devices. arXiv:2512.15491.

[9] William R. Thompson. 1933. On the Likelihood that One Unknown Probability Exceeds Another in View of the Evidence of Two Samples. *Biometrika* 25, 3/4, 285-294.

[10] R. William Soukoreff and I. Scott MacKenzie. 2003. Metrics for Text Entry Research. In *Proceedings of CHI '03*. ACM, 113-120.

[11] Jacob O. Wobbrock, Shaun K. Kane, Krzysztof Z. Gajos, Susumu Harada, and Jon Froehlich. 2011. Ability-Based Design: Concept, Principles and Examples. *ACM Transactions on Accessible Computing* 3, 3.

[12] Xiaoyi Zhang, Harish Kulkarni, and Meredith Ringel Morris. 2017. Smartphone-Based Gaze Gesture Communication for People with Motor Disabilities. In *Proceedings of CHI '17*. ACM, 2878-2889.